

INFORME DE AUDITORÍA DE SEGURIDAD

BootWhatsapp

CRM y Bot de WhatsApp multi-empresa · Backend Django/DRF

Fecha	8 de julio de 2026
Clasificación	CONFIDENCIAL
Alcance	Backend Django/DRF, API gateway, webhook Meta, pipeline LLM, despliegue nativo Windows, secretos y criptografía
Metodología	Revisión de código en 5 dimensiones + verificación manual + remediación aplicada y probada
Base de código	~17.300 LOC (app chat) · repos bootwhatsapp + plataform_site

Veredicto general. Base madura y bien endurecida. **Cero vulnerabilidades críticas o altas de código.** Todos los defectos de código detectados (4 medios · 7 bajos) fueron **corregidos, probados y verificados** en esta ronda (105 tests OK, Django check limpio). El único riesgo abierto es **operativo** (rotación de secretos), que depende de acción del operador en los proveedores.

REMEDIACIÓN APLICADA · 11/11 HALLAZGOS DE CÓDIGO RESUELTOS

Documento generado para uso interno. Contiene rutas de código y descripciones de vectores de ataque; trátese como material sensible y no se distribuya fuera del equipo autorizado.

1. Resumen ejecutivo

Se auditó el estado actual del código sobre cinco dimensiones de seguridad, cada una revisada de forma independiente y con verificación manual de los hallazgos de mayor impacto. A diferencia de una auditoría de solo diagnóstico, en esta ronda **se aplicaron y probaron las correcciones** de todos los defectos de código.

11DEFECTOS DE CÓDIGO
CORREGIDOS**0**CRÍTICO/ALTO DE
CÓDIGO**1**OPERATIVO ABIERTO
(SECRETOS)**105**

TESTS OK TRAS EL FIX

- **Cero** vulnerabilidades críticas o altas *de código* explotables externamente, antes y después de la ronda.
- **Los 4 hallazgos medios y los 7 bajos de código quedaron resueltos**, verificados con la suite completa (105 tests) y el chequeo de sistema de Django.
- El único riesgo **crítico es operativo** (C1): rotar los tokens de terceros y proteger el `.env` en disco. No es código; requiere acción en Meta/Groq/Together y en el sistema de archivos del servidor.
- **git limpio** en el historial completo de ambos repositorios: nunca se versionó un `.env` real, `data_backup.json` ni `deploy.config.ps1`.

2. Registro de hallazgos y estado

ID	Severidad	Área	Título	Estado
C1	CRÍTICO	Secretos	Secretos reales de producción en <code>.env</code> en disco; tokens de terceros sin rotar	Acción operador
M1	MEDIO	Multi-tenant	<code>get_active_connection()</code> sin filtro de empresa → ruteo cross-tenant en fallback	✓ Corregido
M2	MEDIO	RBAC	4 endpoints de Reportes/Super-Casos con <code>IsAuthenticated</code> en vez de <code>section_perm</code>	✓ Corregido
M3	MEDIO	Config/TLS	<code>--forwarded-proto https</code> + bind <code>0.0.0.0</code> sin TLS real en LAN	✓ Corregido
M4	MEDIO	Red	Whitelist de IP del engine opt-in e inerte por defecto	✓ Corregido
B1	BAJO	Config	JWT sin rotación/revocación de refresh tokens (sin blacklist)	✓ Corregido
B2	BAJO	DoS	<code>download_meta_media</code> sin <code>stream</code> ni tope de bytes	✓ Corregido
B3	BAJO	SSRF	<code>download_url</code> de Meta sin allowlist de host	✓ Corregido
B5	BAJO	Despliegue	PAT embebido en URL de <code>git</code> (visible en args de proceso)	✓ Corregido

ID	Severidad	Área	Título	Estado
B6	BAJO	DoS	serve.py lee el body proxeadado completo sin tope	✓ Corregido
B7	BAJO	LLM	Turno de usuario en vivo sin re-sanitizar; regex de secretos con huecos	✓ Corregido
B4	INFO	Rate-limit	Rate-limit del gateway por-worker (LocMem): requiere cache compartido (infra)	Acción infra
B8	INFO	Auth	Auto-registro público: decisión de negocio	Decisión

✓ **Corregido** = fix aplicado al código y verificado en esta ronda. **Acción operador/infra/decisión** = no es un defecto de código; requiere una acción fuera del repositorio (rotar secretos en el proveedor, configurar un cache compartido, o una decisión de producto).

3. Correcciones aplicadas en esta ronda

Cada defecto de código se corrigió, se compiló y se probó. La suite completa (105 tests, incluidas las 5 suites anti-fugas) pasa en verde y el chequeo de sistema de Django no reporta issues.

MEDIO
M1 · Ruteo de salida cross-tenant en el fallback de conexión
✓ Corregido

UBICACIÓN
chat/services/connection_manager.py:63 · chat/services/message_handler.py:1480

RIESGO
La respuesta a un cliente podía salir por la línea de WhatsApp de otro inquilino cuando la conexión se resolvía por el fallback global (sin `connection_id` / `phone_number_id`).

FIX APLICADO
`get_active_connection()` ahora acepta `company=` y filtra el queryset por empresa. El call site deriva la empresa de la conversación y, si no hay empresa resoluble, **descarta** en lugar de elegir una conexión global (mismo criterio que el webhook Meta).

MEDIO**M2 · Bypass de RBAC de sección en 4 endpoints****✓ Corregido****UBICACIÓN**`chat/crm_views.py` — líneas 8659, 8716, 9106, 9151**RIESGO**

Un usuario sin la sección habilitada (validador, inventario, o agente con la sección quitada) podía obtener reportes agregados y el consolidado del Super-Caso por API directa. Escalada de sección intra-tenant.

FIX APLICADO

`SuperCaseReportView` y `SuperCaseReportExportView` pasan a `section_perm('super_cases')` ; `NaturalLanguageReportView` y `QueryDataView` a `section_perm('reports')` , alineándose con sus vistas hermanas.

MEDIO**M3 · Incoherencia TLS del proxy****✓ Corregido****UBICACIÓN**`native/local.ps1` · `native/serve.py`**RIESGO**

El proxy anunciaba `X-Forwarded-Proto: https` de forma incondicional mientras servía HTTP plano en la LAN, provocando cookies `Secure` /HSTS sobre HTTP.

FIX APLICADO

`local.ps1` deriva `--forwarded-proto` del valor real de `ENABLE_HTTPS` del `.env` (no más `https` hardcodeado) y avisa cuando sirve HTTP. `serve.py` ya condicionaba HSTS a `FORWARDED_PROTO == "https"` , por lo que ahora deja de emitirse sobre HTTP.

MEDIO**M4 · Whitelist de IP del engine inerte por defecto****✓ Corregido****UBICACIÓN**`core/settings.py`**RIESGO**

Sin la whitelist configurada, un secret filtrado permitía inyectar mensajes falsos en cualquier inquilino a través del endpoint interno del engine.

FIX APLICADO

En producción (`DEBUG=False`), si `WHATSAPP_ENGINE_ALLOWED_IPS` está vacío se emite un **warning de seguridad ruidoso al arrancar** para que el operador no despliegue con esa defensa inerte. Verificado: el aviso se dispara en el arranque actual.

BAJO B1 · JWT sin rotación ni revocación

✓ Corregido

UBICACIÓN`core/settings.py` (SIMPLE_JWT · INSTALLED_APPS)**FIX APLICADO**

Añadida la app `rest_framework_simplejwt.token_blacklist` con `ROTATE_REFRESH_TOKENS` y `BLACKLIST_AFTER_ROTATION` activados. Migración aplicada a la BD. Ahora un refresh filtrado o un usuario desactivado deja de poder renovar el access token, en vez de seguir vivo hasta 7 días.

BAJO B2 · B3 · Descarga de media de Meta sin tope ni allowlist de host

✓ Corregido

UBICACIÓN`chat/services/whatsapp_api.py`**FIX APLICADO**

La descarga usa `stream=True` con tope de 25 MB (aborta si el `Content-Length` o el acumulado lo superan) y valida que `download_url` apunte a un CDN de Meta (`*.fbcdn.net / facebook.com / whatsapp.net / lookaside.fbsbx.com`) antes de enviarle el token. Cierra el DoS por memoria y el SSRF teórico.

BAJO B5 · PAT embebido en la URL de git

✓ Corregido

UBICACIÓN`deploy.ps1`**FIX APLICADO**

El token ya no se inyecta en la URL (`x-access-token:$token@github.com`). Se pasa como cabecera `Authorization: Basic` efímera por-comando vía `-c http.extraHeader=...`: nunca se persiste en el remote ni se repite en cada `fetch`.

BAJO B6 · Body del proxy sin tope

✓ Corregido

UBICACIÓN`native/serve.py`**FIX APLICADO**

El proxy rechaza con `413` cualquier request con `Content-Length` superior a 32 MB antes de leerlo en memoria.

BAJO B7 · Endurecimiento del pipeline LLM

✓ Corregido

UBICACIÓN

chat/services/ai_service.py

FIX APLICADO

El turno de usuario en vivo se sanitiza (control chars) antes de entrar al prompt, igual que el historial. Añadido un patrón anti-fuga para valores **Fernet crudos** (gAAAAA...) emitidos sin su etiqueta de nombre, cubriendo un hueco del filtro de secretos de salida.

4. Riesgo abierto — requiere acción del operador

CRÍTICO C1 · Secretos reales de producción en .env en disco

Acción operador

UBICACIÓN

C:\BootWhatsapp\.env (fuera de git; copiado también a Backend\bootwhatsapp\.env por deploy.ps1)

DESCRIPCIÓN

Contiene en texto plano el token de Meta, GROQ_API_KEY , TOGETHER_API_KEY , FERNET_KEY , JWT_SIGNING_KEY , DB_PASSWORD y demás secretos. No es un defecto de código (git está limpio); es exposición en el sistema de archivos del servidor.

ESCENARIO

Cualquiera con lectura del filesystem (malware, backup no cifrado, RDP comprometido, robo del equipo) obtiene el token de Meta (suplanta la línea), la clave Fernet (descifra los access_token de la BD) y la firma JWT (forja sesiones).

ACCIÓN REQUERIDA (NO AUTOMATIZABLE DESDE EL CÓDIGO)

- Rotar en el proveedor los tokens de terceros expuestos: Meta (WHATSAPP_API_TOKEN + WHATSAPP_APP_SECRET), GROQ_API_KEY , TOGETHER_API_KEY .
- Restringir la ACL del archivo: icacls .env /inheritance:r /grant:r "<usuario-servicio>:R" .
- Excluir o cifrar .env en los backups. Cifrar/borrar data_backup.json si existe.
- Al rotar FERNET_KEY : FERNET_KEY=NUEVA,VIEJA → python manage.py reencrypt_fernet --apply → dejar solo NUEVA . Nunca regenerar la Fernet sin reencriptar antes (rompe tokens → Meta 401).

B4 (rate-limit del gateway por-worker) y **B8** (auto-registro público) también quedan como acciones fuera del código: B4 requiere apuntar el cache default a Redis/Memcached compartido en la infraestructura; B8 es una decisión de producto (mantener o no el alta pública). Ambos son de bajo riesgo.

5. Controles verificados correctos

Controles que la auditoría confirmó bien implementados y que deben preservarse en futuras modificaciones.

Autenticación y multi-tenant

- Aislamiento multi-tenant consistente: `get_object_or_404(..., company=_get_company(user))` en todos los accesos por pk; `_get_company` es **fail-closed** (403).
- IDOR protegido (empresa y propiedad). Anti-escalada en «Mi Equipo» (un no-admin no crea/edita/asciende admins).
- Override de permisos **autoritativo**: las claves ausentes se guardan `False`, cerrando el default generoso del rol agente.

Gateway, webhook y firmas HMAC

- Firma HMAC del webhook **fail-closed** + `hmac.compare_digest`. Fallback cross-tenant del webhook eliminado.
- Gateway: hash `sha256(token+salt)`, comparación constant-time, scopes por endpoint, tenant fail-closed.
- Engine secret constant-time; `_trusted_client_ip` resistente a spoofing de `X-Forwarded-For`.

SSRF, media y generación de PDF

- `external_lookup`: allowlist por `ip.is_global`, anti DNS-rebinding real, no sigue redirects, quita el header de auth al cambiar de host, tope de 1 MB.
- Media firmada (path+company, TTL 6h) + anti path-traversal + `as_attachment` + `nosniff` → XSS almacenado neutralizado.
- PDF: todo texto no confiable pasa por `_esc()` antes de `Paragraph` → sin inyección de markup reportlab.

Secretos, criptografía y despliegue

- `SECRET_KEY` sin default; `JWT_SIGNING_KEY` dedicada; `DEBUG=False` por defecto; `ALLOWED_HOSTS` sin comodín; CORS/CSRF fail-closed.
- Fernet: `MultiFernet` con rotación; `from_db_value` fail-safe; `reencrypt_fernet` idempotente.
- waitress bindea a `127.0.0.1`; **git limpio** en historial completo; sin secretos hardcoded en código.

Bot / LLM

- **Portero doble**: `guard_outbound` en cada call site y en `send_with_fallback` → todo texto saliente se filtra.
- Contexto del LLM inyecta solo el nombre del cliente; sin notas internas, `datos_extra` ni liquidaciones. Centinela `_confidencial` en liquidaciones.
- Sin SQL crudo ni `eval` / `exec` / `subprocess` / `pickle` / `yaml.load` con datos de usuario. Suite de 105 tests con 5 archivos anti-fugas.

6. Verificación de la remediación

Comprobación	Resultado
Compilación de todos los archivos Python modificados (<code>py_compile</code>)	✓ OK
Sintaxis de los scripts PowerShell (<code>deploy.ps1</code> , <code>local.ps1</code>)	✓ OK

Comprobación	Resultado
Chequeo de sistema de Django (<code>manage.py check</code>)	✓ 0 issues
Suite de tests completa (<code>manage.py test chat</code>)	✓ 105 OK
Suites anti-fugas y de seguridad (5 archivos, 70 tests)	✓ 70 OK
Migración de blacklist de JWT aplicada a la BD	✓ OK
Warning de seguridad M4 se dispara al arrancar sin whitelist	✓ Verificado
Sin migraciones de <code>chat</code> pendientes (<code>makemigrations --check</code>)	✓ Sin cambios

Metodología. Revisión estática de código en cinco dimensiones independientes (autenticación/RBAC/multi-tenant · gateway/webhook/HMAC · SSRF/media/lookups externos · secretos/criptografía/despliegue · inyección y fugas del pipeline LLM), con verificación manual por lectura directa del código de los hallazgos M1 y M2, seguida de la remediación de todos los defectos de código y su verificación mediante compilación, chequeo de sistema y ejecución de la suite completa de tests. Alcance limitado a análisis estático y pruebas automatizadas; no incluye pruebas dinámicas de penetración. El único riesgo abierto (C1, y los informativos B4/B8) requiere acción operativa/de infraestructura/de negocio fuera del repositorio. — Informe generado el 8 de julio de 2026.